CI601 The Computing Project Report

PHISHGUARD – AI POWERED BROWSER EXTENSION

Lukasz Lapinski (student no.22832736)

University of Brighton, Computing Division

Table of Contents

1. Motivation

In the rapidly evolving digital landscape, the internet has become an indispensable part of daily life, enabling communication, commerce, education, and entertainment on a global scale. However, this increased connectivity has also given rise to a surge in cyber threats, with phishing attacks standing out as one of the most prevalent and damaging forms of online crime. Phishing involves the use of deceptive websites and messages to trick users into divulging sensitive information such as passwords, credit card numbers, and personal data. These attacks are not only widespread but also increasingly sophisticated, often bypassing traditional security measures and exploiting human psychology to achieve their malicious goals.

Despite significant advancements in cybersecurity, phishing remains a persistent challenge for individuals, businesses, and organizations of all sizes. Traditional anti-phishing solutions, such as static blacklists, basic spam filters, and user education campaigns, are often insufficient in the face of rapidly changing attack vectors. Blacklists can only protect against known threats and are easily circumvented by attackers who frequently change domains and URLs. User education, while important, cannot fully compensate for the ever-evolving tactics employed by cybercriminals, who continuously refine their methods to appear more legitimate and convincing.

The motivation for developing PhishGuard arises from the urgent need for a more robust, adaptive, and user-centric approach to phishing detection and prevention. PhishGuard is designed as a browser extension that leverages advanced machine learning techniques, specifically a stacking ensemble of models, to analyse the structure of URLs, the content of web pages, and metadata (included in html files of the websites) in real time. This multi-layered approach enables PhishGuard to detect not only known phishing sites but also novel and previously unseen threats by identifying suspicious patterns, behaviors, and attempts at brand impersonation.

A key driver behind PhishGuard is the desire to make cutting-edge cybersecurity accessible and effective for everyday users. Many existing solutions are either too technical for the average person or require significant manual intervention, which can lead to user fatigue and decreased vigilance. PhishGuard addresses this gap by providing automatic, real-time protection that operates seamlessly in the background, delivering clear and non-intrusive warnings when a potential threat is detected. The extension also offers detailed explanations for its decisions, empowering users to understand the risks and make informed choices about their online activities.

Another important motivation is the commitment to continuous improvement and community involvement. PhishGuard includes a feedback mechanism that allows users to report false positives and negatives, contributing to the ongoing refinement of the detection models. This collaborative approach not only enhances the accuracy of the system but also fosters a sense of shared responsibility in the fight against phishing.

The motivation for PhishGuard is rooted in the recognition that effective phishing protection requires a combination of advanced technology, user-centric design, and a strong commitment to privacy and continuous improvement. By bridging the gap between state-of-the-art machine learning research and practical, accessible tools, PhishGuard aims to provide robust, real-time defence against one of the most pressing cybersecurity threats of our time, ultimately making the internet a safer place for everyone.

2. Aims and objectives

The development of PhishGuard was guided by a set of ambitious aims and objectives designed to address the complex challenge of phishing detection in a user-friendly and privacy-conscious manner. At the outset, the project sought to integrate advanced artificial intelligence, robust data handling, and accessible design into a cohesive browser extension. However, as the project progressed, certain objectives were adapted in response to technical limitations, computational constraints, and the natural learning curve encountered during development. This section outlines the core aims and objectives that shaped PhishGuard, reflecting both the initial vision, and the practical adjustments made along the way.

2.1. Development and Training of the AI Model

Design and train a machine learning models capable of accurately identifying malicious and potentially dangerous websites, utilizing a diverse dataset containing URLs, website content, and relevant metadata. The initial plan was to create and train three base models, each of them scanning different website's data for any suspicious patterns. By utilizing stacking ensemble approach, the plan was to train one more model which aim was to analyse the output of all three base models.
After developing and training models, validate their performance using established metrics such as accuracy, precision, recall, and F1 score to ensure reliable detection capabilities.

2.2. Creation of the Browser Extension's Frontend and Backend

Aim for developing the frontend was to create an intuitive and user-friendly interface that provides clear, discreet indications of website safety. This includes a compact pop-up for quick safety checks and a more detailed view for users seeking technical insights. The interface aims to be accessible to a wide audience, incorporating features such as text resizing, high-contrast modes, zoom capabilities, and potential language support. In terms of backend, the aim is to implement an efficient backend system that seamlessly integrates the browser environment with the AI model, enabling real-time analysis of websites as users browse.

2.3. Dataset Collection and Pre-processing

Gather comprehensive datasets of both safe and malicious websites for all 3 of base models. Ensuring a balanced and representative sample for comprehensive and accurate model training. Pre-process the data to extract meaningful features and maintain dataset quality, supporting effective training and evaluation.

2.4. Security and Privacy Assurance

In order to prioritize user privacy, ensuring that the extension does not compromise personal data or browsing activity. In addition, the plan includes implementing robust measures for handling sensitive information, adhering to best practices in data security and privacy protection.

2.5. Testing and Debugging

Conduct thorough unit testing of individual software components to verify their correctness and reliability. Integration testing was planned to ensure smooth interaction

between the browser extension and the AI model.

3. Research.

3.1. Similar products available on the market

Several anti-phishing solutions already exist on the market:

- Google Safe Browsing
- Microsoft Defender SmartScreen
- Netcraft
- PhishTank

These tools excel at blocking known phishing sites using large, frequently updated blacklists and offer seamless browser integration for real-time protection. However, most rely heavily on blacklists, making them reactive and less effective against new or rapidly changing phishing attacks. Many do not deeply analyse URL structure or provide clear explanations for their warnings, and some may impact browsing speed or user privacy.

3.2. Datasets

3.2.1. Dataset Acquisition: Challenges and Strategies

The foundation of any machine learning-based phishing detection system is the quality and relevance of its datasets. The process of sourcing and assembling datasets for this project is both time-consuming and instructive, revealing several key challenges:

3.2.2. Phishing Dataset Availability and Limitations

Publicly available phishing datasets, such as those from PhishTank, OpenPhish, and various academic sources, are widely used in research. However, a major limitation quickly becoming apparent: phishing websites are inherently short-lived. By the time datasets are published and accessed, the vast majority of phishing URLs are already offline, taken down by hosting providers, or blacklisted by browsers. This means that while these datasets are valuable for training model on URL patterns they are much less useful for content-based or metadata-based analysis, as the original malicious content is no longer accessible.

3.2.3. Legitimate Dataset Sourcing

In contrast, legitimate URLs are easier to obtain and verify. Datasets of benign URLs can be sourced from web directories, search engine results, or curated lists. However, care must be taken to ensure that these URLs are diverse and representative of real-world web traffic, to avoid introducing bias into the model.

3.2.4. Dataset Diversity and Representativeness:

Another challenge is ensuring that both phishing and legitimate datasets are not only large enough for robust model training, but also diverse in terms of domains, languages, and website types. Over-reliance on a single source or type of URL can lead to overfitting and poor generalization.

3.3. Metadata Collection: Scraping and Its Limitations

3.3.1. Scarcity of Metadata Datasets

While URL-based datasets are common, datasets containing rich metadata (such as SSL certificate details, HTTP headers, server information, and page content) for phishing sites are rare. Most public datasets focus solely on the URL and a binary label (phishing or legitimate).

3.3.2. Attempted Metadata Scraping

To address this gap, collecting metadata was the plan. By programmatically visiting URLs from the phishing datasets and extracting information such as SSL certificate details, HTTP response headers, and page content, I could gain the data that is hard to obtain anywhere in form of a ready dataset. This process involves writing custom scripts to automate HTTP requests and parse responses.

3.3.3. Inaccessibility of Phishing Sites

The overwhelming majority of phishing URLs were already offline, redirected, or blocked by the time of scraping. This made it nearly impossible to retrieve live metadata or content for most entries.

3.3.4. Technical Barriers

Many phishing sites that were still online employed anti-bot protections, CAPTCHAs, or rate-limiting, which prevented automated tools from accessing the site. Others returned incomplete or malformed responses, or triggered SSL errors due to expired or self-signed certificates.

3.4. Cleaning and Pre-processing

Given the limitations and inconsistencies commonly found in raw phishing datasets, extensive data cleaning and pre-processing are universally recognized as essential steps to ensure data quality and model reliability in phishing detection research:

3.4.1. Duplicate Removal

It is standard practice to check for and remove duplicate URLs from datasets. Duplicates can lead to data leakage between training and test sets, resulting in overly optimistic performance estimates and biased evaluation.

3.4.2. Handling Missing and Invalid Data

URLs: Rows containing missing, empty, or malformed URLs are typically excluded, as they cannot be used for feature extraction or model training.

Features: For datasets that include additional features (such as content or metadata), missing values in numeric columns are often imputed using statistical measures like the median, while missing categorical values are replaced with a placeholder (e.g., 'missing'). This approach helps preserve as much data as possible without introducing bias.

Labels: Ensuring that all samples have valid, finite labels is critical. Samples with missing or invalid labels are generally removed, as incorrect labels can severely impact model training and evaluation.

Class Balance:

Phishing datasets are frequently imbalanced, with legitimate URLs vastly outnumbering phishing samples. To address this, researchers often employ techniques such as resampling, synthetic data generation (e.g., SMOTE), or constructing balanced datasets. Maintaining class balance is important

to prevent models from being biased toward the majority class and to improve the reliability of performance metrics.

3.4.3. Feature Engineering

For URL-based phishing detection, a comprehensive set of features is typically engineered from the raw URLs. These may include:

Structural features: Lengths of URL components, number of subdomains, etc.

Lexical features: Presence of suspicious keywords, special characters, or patterns.

Statistical features: Entropy measures, digit ratios, and other statistical properties.

Careful parsing and analysis of each URL are required, and features are usually standardized across training, validation, and test sets to ensure consistency.

Data Splitting:

After cleaning and feature engineering, datasets are commonly split into training, validation, and test sets using stratified sampling to preserve class distributions.
This practice ensures fair model evaluation and supports robust hyperparameter tuning.

3.5. Models and training

3.5.1. Model Selection: Considered and Utilized Approaches

In the field of phishing detection, a variety of machine learning models have been explored, each offering distinct advantages and trade-offs depending on the nature of the data and the requirements of the application. Logistic Regression is frequently used as a baseline due to its simplicity and interpretability, making it suitable for rapid prototyping and as a meta-model in ensemble frameworks. Random Forests, as ensemble methods that aggregate the predictions of multiple decision trees, are widely recognized for their robustness to overfitting, ability to model non-linear relationships, and provision of feature importance metrics. These characteristics make Random Forests a popular choice for tabular data and structured feature sets, such as those derived from URLs.

Gradient boosting frameworks like XGBoost and LightGBM have also gained traction in phishing detection research, particularly for their high predictive accuracy and capacity to handle missing data. However, these models can be computationally intensive, and in some cases, the performance gains over Random Forests may not justify the additional complexity, especially when working with moderately sized datasets.

For large-scale datasets, linear models trained with stochastic gradient descent (SGDClassifier) are sometimes employed due to their scalability and speed, though they may not capture complex feature interactions as effectively as tree-based models. Support Vector Machines (SVMs) are another classical approach, valued for their effectiveness in high-dimensional spaces, but their scalability issues and sensitivity to feature scaling often limit their use in large, real-world phishing datasets.

Deep learning models, such as LSTMs or CNNs, have been proposed for both URL and content analysis, leveraging their ability to automatically learn hierarchical feature representations. However, these approaches typically require large labelled datasets, significant computational resources, and careful tuning, and in many cases, classical models with well-engineered features have demonstrated comparable or superior performance for phishing detection tasks.

In summary, the literature suggests that while a range of models can be considered for phishing detection, Random

Forests and gradient boosting methods are often favored for their balance of accuracy, interpretability, and practicality, particularly when working with structured, feature-rich datasets.

3.5.2. The Role of Ensembling

Ensemble methods, and stacking in particular, are frequently advocated in phishing detection research for their potential to improve generalization and robustness. By combining the predictions of diverse base models—such as those trained on URL features, web content, and metadata—ensembles can capture a broader spectrum of phishing tactics and reduce the risk of systematic errors associated with any single model. Stacking, which employs a meta-model to learn how best to combine base model outputs, is especially popular in machine learning competitions and advanced research for its ability to boost predictive performance.

However, practical challenges often arise when implementing ensemble systems in phishing detection. The short-live nature of phishing sites means that content and metadata are frequently unavailable, limiting the feasibility of content-based and metadata-based models. Integrating diverse models also introduces complexity in feature alignment, data pre-processing, and system maintenance. Furthermore, the computational and operational overhead of training, deploying, and maintaining multiple models can be significant, particularly in resource-constrained environments. As a result, while ensembling remains a promising avenue, many real-world systems ultimately rely on a single, robust model—often based on URL features—due to its reliability, speed, and ease of deployment.

3.5.3. Rationale for Model and Feature Choices

URL-based models are a mainstay in phishing detection research, primarily because URLs are always available, even when the associated website is offline. By engineering a comprehensive set of features—including structural properties (such as the length of URL components and the number of subdomains), lexical cues (like suspicious keywords and special characters), and statistical measures (such as entropy and digit ratios)—researchers can capture a wide array of phishing indicators. While deep learning approaches on raw URLs have been explored, classical models with carefully crafted features often achieve similar or better results, especially when data is limited.

Content-based models, which analyse HTML, JavaScript, and visual elements, are theoretically powerful for detecting phishing sites that mimic legitimate ones despite having benign-looking URLs. However, the practical challenge of acquiring live content from phishing sites—most of which are taken down quickly—limits the applicability of this approach in many studies.

Metadata and brand-based models, leveraging SSL certificate details, HTTP headers, and brand impersonation signals, offer another layer of detection, particularly for sophisticated attacks. Yet, as with content-based methods, the lack of accessible, up-to-date metadata for phishing sites remains a significant barrier.

Other features, such as WHOIS data and domain age, have been considered in the literature, but are often omitted due to API limitations, latency concerns, or data availability. Visual similarity analysis, while promising, is computationally expensive and requires live site access, further constraining its use in large-scale or real-time systems.

3.5.4. Model Training and Evaluation Practices

Best practices in phishing detection research emphasize rigorous data cleaning, removing duplicates, and splitting datasets into training, validation, and test sets. Feature engineering is applied to extract a rich set of characteristics from URLs and, where possible, from content and metadata. Models are typically trained and evaluated using a suite of metrics, including accuracy, precision, recall, F1 score, and confusion matrices, to provide a comprehensive assessment of performance. Hyperparameter tuning and feature importance analysis are also standard, enabling researchers to optimize models and interpret their decisions.

Ultimately, the choice of model is guided by a balance of predictive performance, interpretability, computational efficiency, and practical constraints such as data availability and deployment requirements. In many cases, a well-tuned Random Forest or gradient boosting model trained on engineered URL features emerg es as the most effective and pragmatic solution for real-world phishing detection.

4. Tools Used

4.1. Google Colab

Google Colab was selected as the primary environment for data exploration, pre-processing, and model training. Its appeal lies in several key advantages. Most notably, Colab provides free access to GPUs and TPUs, which is invaluable for training large models or processing big datasets without the need for local hardware investment. The platform requires zero setup, as it comes pre-installed with most scientific Python libraries (such as pandas, numpy, scikit-learn, xgboost, and tensorflow), eliminating the friction of manual environment configuration. Notebooks can be easily shared, commented on, and edited by multiple users, which is particularly useful for team-based research or when seeking feedback. Additionally, Colab's integration with Google Drive allows for straightforward storage and access to large datasets and model files.

Several alternatives were considered. Kaggle Kernels offer a similar notebook interface and free compute, but with stricter resource and time limits, and less flexibility in package management. Local Jupyter Notebooks provide full control and direct access to local files, but require a well-equipped machine and manual dependency management. Cloud-based solutions like AWS SageMaker, Azure Notebooks, or GCP AI Platform offer scalable, production-grade resources, but involve more complex setup and incur costs. Local environments allow for reproducible, isolated setups, but require more technical overhead and local compute resources. Ultimately, Colab's combination of free, powerful compute, ease of use, and collaborative features made it the most practical and efficient choice for iterative development and experimentation, especially in the early and middle stages of the project.

4.2. Visual Studio Code (VS Code)

For code editing, debugging, and project management, Visual Studio Code (VS Code) was the tool of choice. VS Code's strengths include a rich extension ecosystem—supporting Python, Jupyter, Git, Docker, and JavaScript/TypeScript—which streamlines development across the stack. Its integrated terminal and debugging tools enable running scripts, managing environments, and troubleshooting directly within the editor. The cross-

platform support ensures a flexible workflow, and its multi-language capabilities are essential for projects involving both Python (for backend and machine learning) and JavaScript (for browser extension development).Alternatives such as PyCharm offer advanced refactoring and debugging for Python, but are heavier and less flexible for multi-language projects. Editors like Sublime Text and Atom are lightweight and fast, but lack the integrated development and debugging features of VS Code. Vim and Emacs are highly customizable, but have a steeper learning curve and less out-of-the-box support for modern workflows. VS Code's balance of speed, extensibility, and integrated tooling made it ideal for managing a multi-language, multi-component project.

4.3. Python Libraries and Dependencies

A range of well-established Python libraries formed the backbone of the data science and machine learning workflow. The key libraries used (as reflected in requirements.txt and the codebase) include:

- **pandas, numpy:** Core libraries for data manipulation, cleaning, and numerical operations, used extensively for dataset preprocessing and feature engineering.
- **scikit-learn:** The backbone of the machine learning pipeline, providing models (RandomForest, LogisticRegression, SGDClassifier), preprocessing utilities, and evaluation metrics.
- **xgboost:** Used for experimenting with gradient boosting models, especially for content-based and metadata-based approaches.
- **scipy:** Required for scientific computing and as a dependency for scikit-learn and xgboost.
- **tensorflow:** Present for compatibility, but not used in the final pipeline (included for possible deep learning experiments).
- **bottleneck:** Performance booster for pandas operations.
- **requests, beautifulsoup4:** Used for web scraping and metadata extraction—requests for HTTP requests, beautifulsoup4 for HTML parsing.

These libraries are industry standards, well-documented, and highly optimized for data science and machine learning tasks. scikit-learn, in particular, is the de facto standard for tabular ML workflows, while pandas and numpy are essential for any data manipulation. Alternatives such as PyTorch or Keras were not used, as deep learning was not the focus and classical models performed well. CatBoost and LightGBM were considered, but xgboost and RandomForest provided sufficient performance for the project's needs. More advanced scraping libraries (e.g., Scrapy) were not necessary for the relatively simple metadata extraction tasks.

4.4. JavaScript vs. React for the Browser Extension

The browser extension was implemented using plain JavaScript, rather than a framework like React. This decision was driven by the simplicity of the extension's UI and logic, which did not require the complexity of a component-based framework. Using plain JavaScript ensures minimal bundle size and fast load times, which is important for browser extensions that need to be lightweight and responsive. Debugging is also simpler without the abstraction layers and build steps introduced by React, and vanilla JavaScript is universally supported and integrates easily with browser APIs.

React is powerful for building complex, interactive web applications, but would have added unnecessary overhead (build tools, dependencies, larger bundles) for this project. The extension's requirements were met efficiently with standard JavaScript, HTML, and CSS.

4.5. FastAPI for the Backend API

FastAPI was chosen as the backend framework for serving the phishing detection model. Its advantages include high performance (built on Starlette and Uvicorn, with asynchronous support for handling many requests per second), automatic OpenAPI (Swagger) documentation for easy testing and documentation, and type safety and validation via Python type hints and Pydantic. FastAPI's modern syntax is clean, concise, and easy to maintain, with strong community support.

Alternatives such as Flask are simpler and widely used, but lack built-in async support and automatic documentation. Django REST Framework is powerful and feature-rich, but overkill for a lightweight API and more complex to set up. Node.js/Express is a good choice for JavaScript stacks, but Python was preferred for seamless ML model integration. FastAPI offered the best combination of speed, modern features, and ease of integration with Python-based machine learning models, making it the optimal choice for this project.

5. Development and Implementation

The development and implementation of the PhishGuard project followed an iterative, research-driven process, shaped by practical challenges, experimental results, and evolving project goals. This section details the major phases of implementation, technical decisions, and the integration of system components.

5.1. Initial Setup and Environment

The project began with the selection of tools and environments that would support both rapid prototyping and robust development. Google Colab was chosen for its free access to powerful compute resources, collaborative features, and pre-installed scientific libraries, making it ideal for data exploration, cleaning, and model training. Visual Studio Code (VS Code) was used as the primary code editor, offering a rich extension ecosystem, integrated terminal, and support for both Python and JavaScript, which was essential for managing the backend, machine learning scripts, and browser extension code in a unified environment.

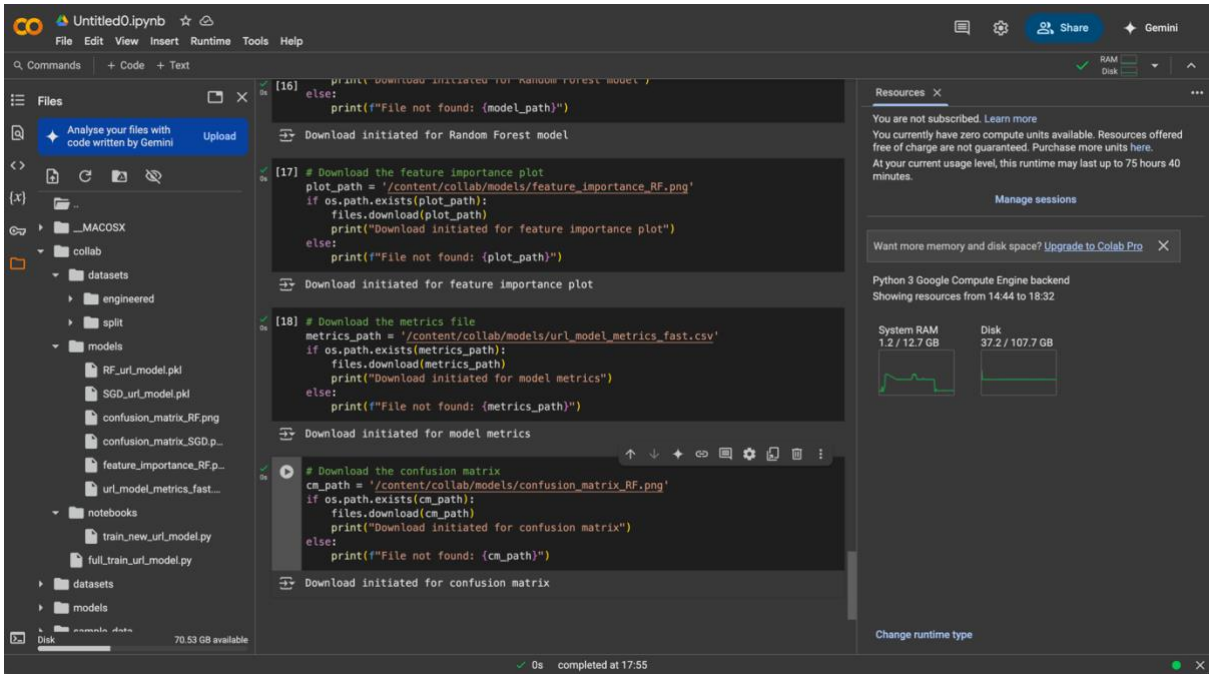


Figure 1: Google Colab

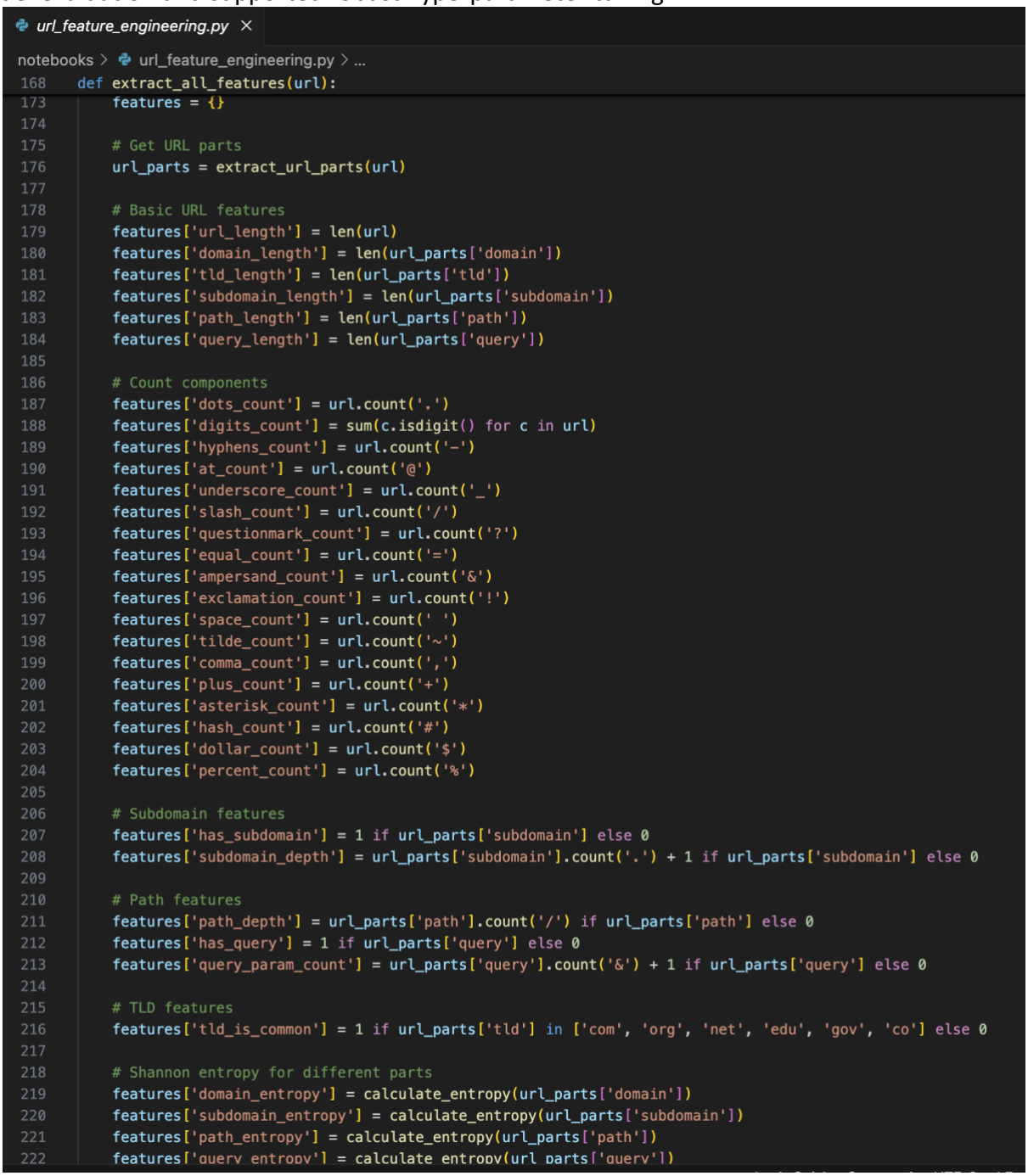
5.2. Data Acquisition and Preprocessing

A critical early phase involved sourcing and preparing datasets. Multiple public datasets were collected, including those containing phishing and legitimate URLs, as well as feature-rich datasets like PhiUSIIL_Phishing_URL_Dataset.csv. The process quickly revealed a major challenge: the majority of phishing URLs in public datasets were already inactive, making it nearly impossible to collect up-to-date content or metadata for these sites. This limitation significantly influenced subsequent design decisions.

To ensure data quality and model reliability, a robust data cleaning and preprocessing pipeline was developed.
All datasets were checked for and purged of duplicate URLs to prevent data leakage and ens ure unbiased evaluation. Rows with missing, empty, or malformed URLs were removed, as these could not be used for feature extraction or model training.
For datasets with additional features,
missing values in numeric columns were imputed with the median,
while missing categorical values were replaced with a placeholder to preserve as much data as possible without introducing bias. Special care was taken to ensure that all samples had valid, finite labels, as incorrect labels can severely impact model performance.

Class imbalance, a common issue in phishing datasets, was addressed by constructing a new, balanced dataset (final_url_dataset.csv) with roughly equal numbers of phishing and legitimate URLs. This step was crucial for preventing the model from being biased towards the majority class and for improving the reliability of performance metrics.Feature engineering was a major focus, particularly for the URL-based model. A comprehensive set of features was engineered from the raw URLs,
including structural properties (such as lengths of URL components and number of subdomains), lexical cues (like suspicious keywords and special characters),
and statistical measures (such as entropy and digit ratios). This process required careful parsing and analysis of each URL, and the resulting features
were standardized across training, validation, and test sets to ensure consistency.

After cleaning and feature engineering, datasets were split into training, validation, and test sets using stratified sampling to preserve class distributions. This practice ensured fair model evaluation and supported robust hyperparameter tuning.



```python
# url_feature_engineering.py
# notebooks > url_feature_engineering.py > ...
def extract_all_features(url):
    features = {}

    # Get URL parts
    url_parts = extract_url_parts(url)

    # Basic URL features
    features['url_length'] = len(url)
    features['domain_length'] = len(url_parts['domain'])
    features['tld_length'] = len(url_parts['tld'])
    features['subdomain_length'] = len(url_parts['subdomain'])
    features['path_length'] = len(url_parts['path'])
    features['query_length'] = len(url_parts['query'])

    # Count components
    features['dots_count'] = url.count('.')
    features['digits_count'] = sum(c.isdigit() for c in url)
    features['hyphens_count'] = url.count('-')
    features['at_count'] = url.count('@')
    features['underscore_count'] = url.count('_')
    features['slash_count'] = url.count('/')
    features['questionmark_count'] = url.count('?')
    features['equal_count'] = url.count('=')
    features['ampersand_count'] = url.count('&')
    features['exclamation_count'] = url.count('!')
    features['space_count'] = url.count(' ')
    features['tilde_count'] = url.count('~')
    features['comma_count'] = url.count(',')
    features['plus_count'] = url.count('+')
    features['asterisk_count'] = url.count('*')
    features['hash_count'] = url.count('#')
    features['dollar_count'] = url.count('$')
    features['percent_count'] = url.count('%')

    # Subdomain features
    features['has_subdomain'] = 1 if url_parts['subdomain'] else 0
    features['subdomain_depth'] = url_parts['subdomain'].count('.') + 1 if url_parts['subdomain'] else 0

    # Path features
    features['path_depth'] = url_parts['path'].count('/') if url_parts['path'] else 0
    features['has_query'] = 1 if url_parts['query'] else 0
    features['query_param_count'] = url_parts['query'].count('&') + 1 if url_parts['query'] else 0

    # TLD features
    features['tld_is_common'] = 1 if url_parts['tld'] in ['com', 'org', 'net', 'edu', 'gov', 'co'] else 0

    # Shannon entropy for different parts
    features['domain_entropy'] = calculate_entropy(url_parts['domain'])
    features['subdomain_entropy'] = calculate_entropy(url_parts['subdomain'])
    features['path_entropy'] = calculate_entropy(url_parts['path'])
    features['query_entropy'] = calculate_entropy(url_parts['query'])
```

Figure 2: Feature engineering in the process of data cleaning and pre-processing

5.3. Model Development and Experimentation

The model development phase was highly exploratory, involving the evaluation of several machine learning algorithms. Logistic Regression was initially used for its simplicity and interpretability, serving as a baseline and for quick prototyping. Random Forests were extensively tested and ultimately became the primary model due to their robustness to overfitting, ability to handle non-linear relationships, and provision of

feature importance metrics. Gradient boosting frameworks such as XGBoost were also considered, particularly for content-based and metadata-based models, but were not used in the final deployment due to computational constraints and diminishing returns compared to Random Forests.

Other models, such as SGDClassifier and Support Vector Machines, were evaluated for their scalability and effectiveness in high-dimensional spaces, but were ultimately not selected due to either lower performance or practical limitations with large datasets. Deep learning approaches, including LSTMs and CNNs, were considered in the research phase but not pursued further due to the lack of sufficiently large labeled datasets, higher computational requirements, and the strong performance of classical models with engineered features.

The initial plan included the use of a stacking ensemble, combining the strengths of multiple base models (URL-based, content-based, and metadata-based) with a meta-model to make the final prediction. However, practical challenges—such as the inaccessibility of live content and metadata, integration complexity, and increased computational overhead—led to a focus on a single, robust URL-based model for the final deployment.

The final model was trained on the engineered URL features, with hyperparameters tuned for optimal performance. The training pipeline included loading and aligning feature sets, splitting data into training, validation, and test sets, training the model, and evaluating it using metrics such as accuracy, precision, recall, F1 score, and confusion matrices. The URL model scored nearly 99% in accuracy, precision and recall. The content model also scored high, metadata model was not very precise due to lack of suitable data. Although due to compatibility issues mentioned above, only the URL model was utilised in the final product.

train_new_url_model.py

notebooks > train_new_url_model.py > ...

```
            if len(unique) > 1 and min(counts) / max(counts) < 0.8:
                print("Applying SMOTE to balance classes...")
                smote = SMOTE(random_state=42)
                X_train_clean, y_train_clean = smote.fit_resample(X_train_clean, y_train_clean)
                print(f"Shape after SMOTE: {X_train_clean.shape}")
        except Exception as e:
            print(f"SMOTE failed: {str(e)}. Continuing with original data.")

        # Train both LogisticRegression and RandomForest models
        print("\n===== Training Logistic Regression =====")
        lr_model, lr_metrics = train_and_evaluate_model(
            X_train_clean, y_train_clean,
            X_val_clean, y_val_clean,
            model_type='LogisticRegression'
        )

        print("\n===== Training Random Forest =====")
        rf_model, rf_metrics = train_and_evaluate_model(
            X_train_clean, y_train_clean,
            X_val_clean, y_val_clean,
            model_type='RandomForest'
        )

        # Compare models
        if lr_metrics and rf_metrics:
            print("\n===== Model Comparison =====")
            metrics_df = pd.DataFrame({
                'LogisticRegression': lr_metrics,
                'RandomForest': rf_metrics
            })
            print(metrics_df)

            # Save metrics
            metrics_path = os.path.join(models_dir, 'url_model_metrics.csv')
            metrics_df.to_csv(metrics_path)
            print(f"Saved model comparison metrics to {metrics_path}")

        print("\nURL model training complete!")

    except Exception as e:
        print(f"An error occurred: {str(e)}")
        import traceback
        traceback.print_exc()
```

Figure 3: part of URL model training

train_content_model.py

notebooks > train_content_model.py > ...

```python
def main():
        print(f"Training with {len(common_features)} common features")
        X_train_clean = X_train_clean[common_features]
        X_val_clean = X_val_clean[common_features]

        # Check class balance
        unique_train, counts_train = np.unique(y_train_clean, return_counts=True)
        class_distribution = dict(zip(unique_train, counts_train))
        print(f"Class distribution in training set: {class_distribution}")

        # Train both Random Forest and XGBoost models
        rf_metrics = train_and_evaluate_model(
            X_train_clean, y_train_clean,
            X_val_clean, y_val_clean,
            model_type='RandomForest'
        )

        xgb_metrics = train_and_evaluate_model(
            X_train_clean, y_train_clean,
            X_val_clean, y_val_clean,
            model_type='XGBoost'
        )

        # Compare models
        print("\n===== Model Comparison =====")
        metrics_df = pd.DataFrame({
            'RandomForest': rf_metrics,
            'XGBoost': xgb_metrics
        })
        print(metrics_df)

        # Save metrics
        metrics_path = os.path.join(models_dir, 'content_model_metrics.csv')
        metrics_df.to_csv(metrics_path)
        print(f"Saved model comparison metrics to {metrics_path}")

        print("\nContent-based model training complete!")

        # Return the best model based on F1 score
        if rf_metrics['f1'] > xgb_metrics['f1']:
            print("\nRandom Forest model performed better based on F1 score")
            return 'RandomForest'
        else:
            print("\nXGBoost model performed better based on F1 score")
            return 'XGBoost'
```

Figure 4: part of content model training

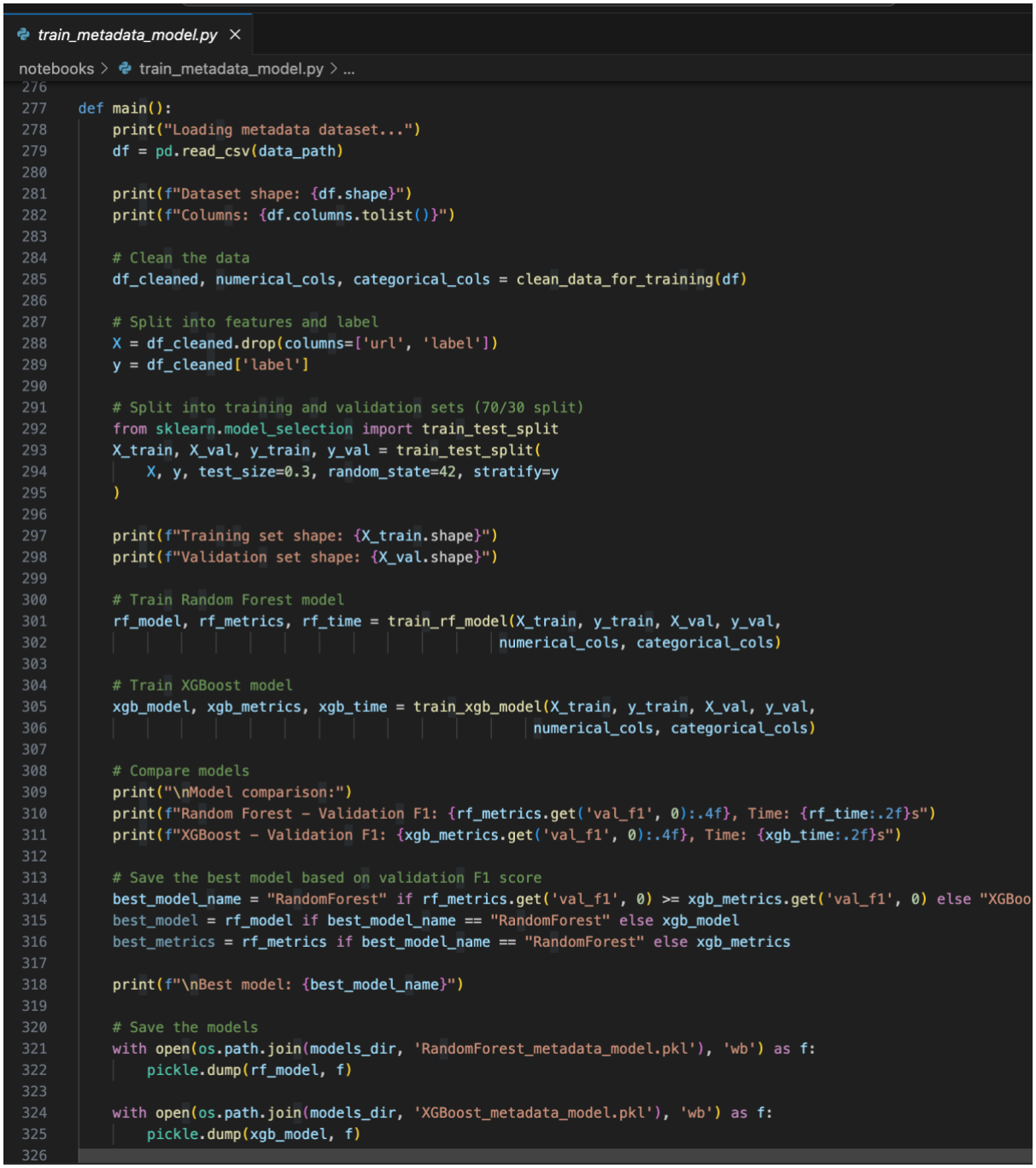


Figure 5: part of metadata model training

5.4. Backend API Development

The backend API was implemented using FastAPI, selected for its high performance, asynchronous support, and automatic documentation generation. The API was designed to load the trained model at startup and expose endpoints for URL analysis and user feedback. The main prediction endpoint receives a URL, extracts features, and returns a phishing prediction along with a confidence score and feature contributions. The API also includes endpoints for health checks and user feedback, supporting monitoring and future model improvements.

Integration with the machine learning model was achieved through a modular design, allowing for easy updates or replacement of the model as needed. The API was tested for performance, reliability, and correctness, ensuring it could handle real-time requests from the browser extension.

5.5. Browser Extension Development

The browser extension was developed using plain JavaScript, HTML, and CSS, adhering to Chrome Manifest V3 requirements. The extension retrieves the current tab's URL, sends it to the backend API, and displays the result to the user with a clear status and confidence score. The user interface was designed to be intuitive and responsive, providing immediate feedback and allowing users to report misclassifications. The extension was tested across browsers for compatibility and responsiveness, ensuring a seamless user experience.

Figure 6: part of JS script for the browser extension

5.6. Integration and System Testing

A key phase of the project involved integrating the browser extension, backend API, and trained model into a cohesive system. This required careful coordination of data formats, API endpoints, and error handling to ensure smooth communication between

components. The system was tested end-to-end to verify real-time phishing detection, user feedback handling, and API health monitoring. Extensive testing was conducted to identify and resolve issues related to latency, error propagation, and edge cases.

5.7. Documentation, Reproducibility, and Version Control

Throughout the project, all scripts, notebooks, and code were maintained in a version-controlled repository. Each step of the process—from data cleaning to deployment—was thoroughly documented to ensure transparency and reproducibility. Intermediate datasets, engineered features, and trained models were saved and versioned, facilitating future development, research, and potential handover to other teams.

5.8. Iterative Improvements and Lessons Learned

The development process was highly iterative, with frequent pivots in response to data limitations, model performance, and integration challenges. Early ambitions for a stacking ensemble were scaled back in favour of a robust, deployable URL-based model. This pragmatic approach ensured a reliable, maintainable system while laying the groundwork for future enhancements as more comprehensive datasets become available. The project underscored the importance of flexibility, thorough data validation, and modular design in building real-world machine learning systems.

6. Testing and Evaluation

Thorough testing and evaluation were integral to the development of the PhishGuard system, ensuring both the reliability of the machine learning model and the robustness of the integrated application. This section outlines the strategies, metrics, and procedures used to assess the system's performance at multiple levels.

6.1. Model Evaluation

6.1.1. Data Splitting and Validation

To ensure unbiased assessment of model performance, the cleaned and feature-engineered dataset was split into training, validation, and test sets using stratified sampling. This preserved the original class distribution across all splits, which is critical for reliable evaluation in the presence of class imbalance.

6.1.2. Evaluation Metrics

The model was evaluated using a comprehensive set of metrics, including:

- **Accuracy:** The proportion of correctly classified URLs.
- **Precision:** The proportion of predicted phishing URLs that were actually phishing (minimizing false positives).

- **Recall:** The proportion of actual phishing URLs that were correctly identified (minimizing false negatives).
- **F1 Score:** The harmonic mean of precision and recall, providing a balanced measure for imbalanced datasets.
- **Confusion Matrix:** A detailed breakdown of true positives, true negatives, false positives, and false negatives, offering insight into the types of errors made by the model.

These metrics were calculated on both the validation and test sets to monitor for overfitting and to estimate real-world performance.

	A	B	C
1		SGD	RandomForest
2	Accuracy	0.87053009	0.91528078
3	Precision	0.87620104	0.88423225
4	Recall	0.87924081	0.96618916
5	F1	0.87771829	0.92339573
6			

Figure 7: URL model score

6.1.3. Feature Importance and Interpretability

Feature importance analysis was conducted to understand which engineered URL features contributed most to the model's decisions. This not only provided transparency but also guided further feature engineering and model refinement.

6.2. API and Backend Testing

The FastAPI backend was subjected to both unit and integration testing:

- **Unit Testing:** Core functions for feature extraction, model loading, and prediction were tested in isolation to ensure correctness.
- **Integration Testing:** The full API workflow—from receiving a URL to returning a prediction—was tested to verify seamless interaction between components.
- **Performance Testing:** The API was evaluated for response time and throughput under simulated load, ensuring suitability for real-time browser extension use.
- **Error Handling:** The system was tested for resilience to malformed requests, missing data, and model loading failures, with appropriate error messages and fallback mechanisms.

6.3. Browser Extension Testing

The browser extension underwent extensive functional and usability testing:

- **User Interaction:** All user interface elements, including status displays, confidence scores, and feedback/reporting mechanisms, were tested for clarity and responsiveness.
- **API Integration:** The extension's communication with the backend API was validated for reliability, including handling of network errors and API downtime.

- **Edge Cases:** Scenarios such as non-HTTP URLs, local files, and unsupported pages were tested to ensure graceful handling and informative user feedback.

6.4. End-to-End System Testing

End-to-end testing was performed to validate the complete workflow:

- A variety of URLs (phishing, legitimate, and ambiguous) were processed through the extension, API, and model to ensure accurate and timely detection.

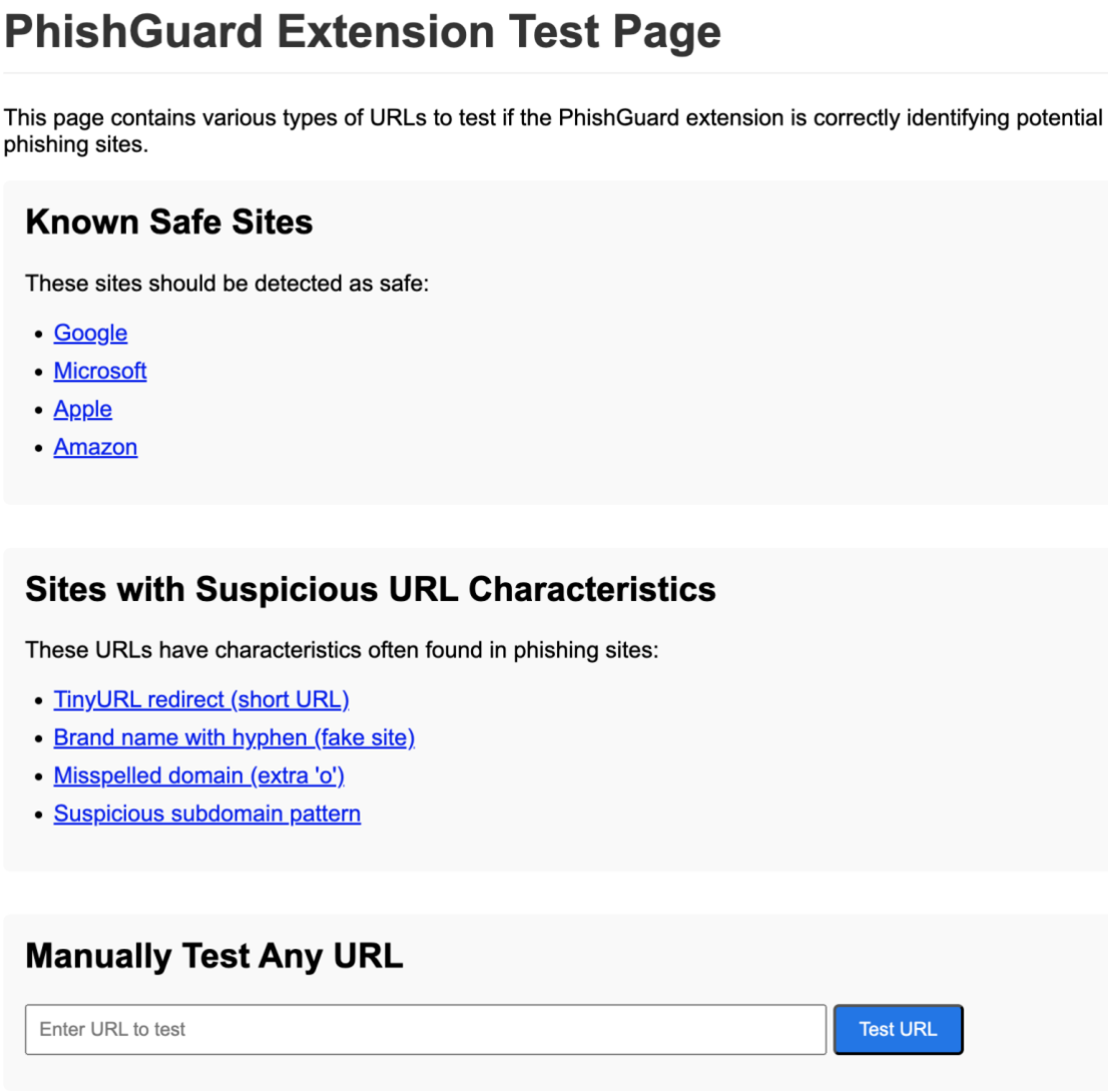


Figure 8: PhishGuard's dedicated testing page

6.5. Limitations and Future Evaluation

While the system demonstrated strong performance on the available test data, certain limitations were acknowledged:

- The inability to evaluate content-based and metadata-based models due to the lack of live phishing sites.

- Potential dataset drift over time, necessitating ongoing monitoring and periodic retraining.
- The need for real-world user feedback and deployment data to further validate and refine the model.
- Improving of user interface and some of the metrics e.g. detection confidence.
- Creating a logo and icons.

  Future evaluation plans include A/B testing in real-world environments, continuous monitoring of false positive/negative rates, and integration of user feedback to support iterative improvement.

7. Final Product

The PhishGuard project was conceived as a comprehensive, modular phishing detection system, integrating multiple machine learning models and data sources to maximize detection accuracy and robustness. The development process involved the design, implementation, and evaluation of several distinct models, each targeting a different aspect of the phishing detection problem.

7.1. Models Developed and Their Intended Roles

URL-Based Model: The URL-based model forms the backbone of the PhishGuard system. This model was built using a Random Forest classifier trained on a large, balanced dataset of phishing and legitimate URLs. Over 60 features were engineered from each URL, capturing structural properties (such as the length of the domain, path, and query), lexical cues (presence of suspicious keywords, special characters, digit ratios), and statistical measures (entropy, Shannon information, etc.). The model was designed for speed and reliability, making it suitable for real-time detection in a browser extension.
Its interpretability also allowed for feature importance analysis, providing transparency into the decision-making process.

Content-Based Model: To complement the URL model, a content-based model was developed using datasets that included HTML, JavaScript, and other page content features. This model leveraged advanced algorithms such as XGBoost and Random Forest to analyze the structure and content of web pages, aiming to detect phishing attempts that closely mimic legitimate sites or use obfuscated URLs. Feature engineering for this model included extraction of DOM properties, script analysis, and detection of suspicious elements within the page content. The content-based model was intended to catch sophisticated attacks that might evade URL-only detection. As mentioned above, this model was developed but due to technical limitations, not used in a final product.

Metadata/Brand-Based Model: A third model focused on metadata and brand signals, utilizing features such as SSL certificate details (issuer, validity, encryption type), HTTP headers, server information, and indicators of brand impersonation. The goal was to identify phishing sites that use technical tricks or target specific brands, which may not be apparent from the URL or page content alone. This model also explored the use of one-hot encoding for categorical metadata and the calculation of time-based features (e.g.,

certificate age, time to verification). As mentioned above, this model was developed but due to technical limitations, not used in a final product.

Ensemble and Stacking Approaches: The original system architecture envisioned a stacking ensemble, where the predictions of the URL, content, and metadata models would be combined using a meta-model (such as Logistic Regression or Random Forest).

This ensemble approach was designed to leverage the strengths of each base model, improving overall accuracy and robustness by capturing a wider range of phishing tactics and reducing the risk of systematic errors.

	A	B	C
1		TunedRando	TunedXGBoost
2	accuracy	0.76546949	0.76575797
3	precision	0.76569038	0.76575797
4	recall	0.99962328	1
5	f1	0.86715686	0.86734194
6	auc	0.75047599	0.7500811
7	tuning_time	1456.00155	77.1017489

Figure 9: content based model score

	A	B	C	D	E	F	G	H	I	J	K
1	model	accuracy	precision	recall	f1	val_accuracy	val_precisior	val_recall	val_f1	val_auc	training_time
2	RandomFore	0.9319407	0.74662162	0.89473684	0.81399632	0.87127159	0.58571429	0.77358491	0.66666667	0.88615286	0.30349183
3	XGBoost	0.96091644	0.86770428	0.90283401	0.88492063	0.88540031	0.63636364	0.72641509	0.6784141	0.85300963	0.11070514

Figure 10: metadata based model score

7.2. Integration and System Design

The backend was implemented using FastAPI, providing endpoints for URL analysis, user feedback, and health monitoring. The browser extension, built with plain JavaScript, interacted with the API to deliver real-time phishing detection and user alerts. The system was designed to be modular, allowing for the addition or replacement of models as new data sources or detection techniques became available.

7.3. The Final Working Product

Due to limitations, the final deployed version of PhishGuard operates exclusively with the URL-based model. The content-based and metadata-based models, while developed and tested in isolation, could not be reliably deployed due to the lack of live data and integration challenges. The stacking ensemble was also set aside in favour of a streamlined, robust solution.

The final product thus consists of:

- A highly optimized URL-based Random Forest model, capable of real-time phishing detection using only the URL.
- A FastAPI backend that serves the model and handles user feedback.
- A browser extension that provides immediate, user-friendly phishing alerts.

The system remains extensible, with the infrastructure in place to incorporate additional models in the future as data and resources permit. While the final product is more focused than originally envisioned, it delivers strong, reliable protection and a solid foundation for future enhancements.

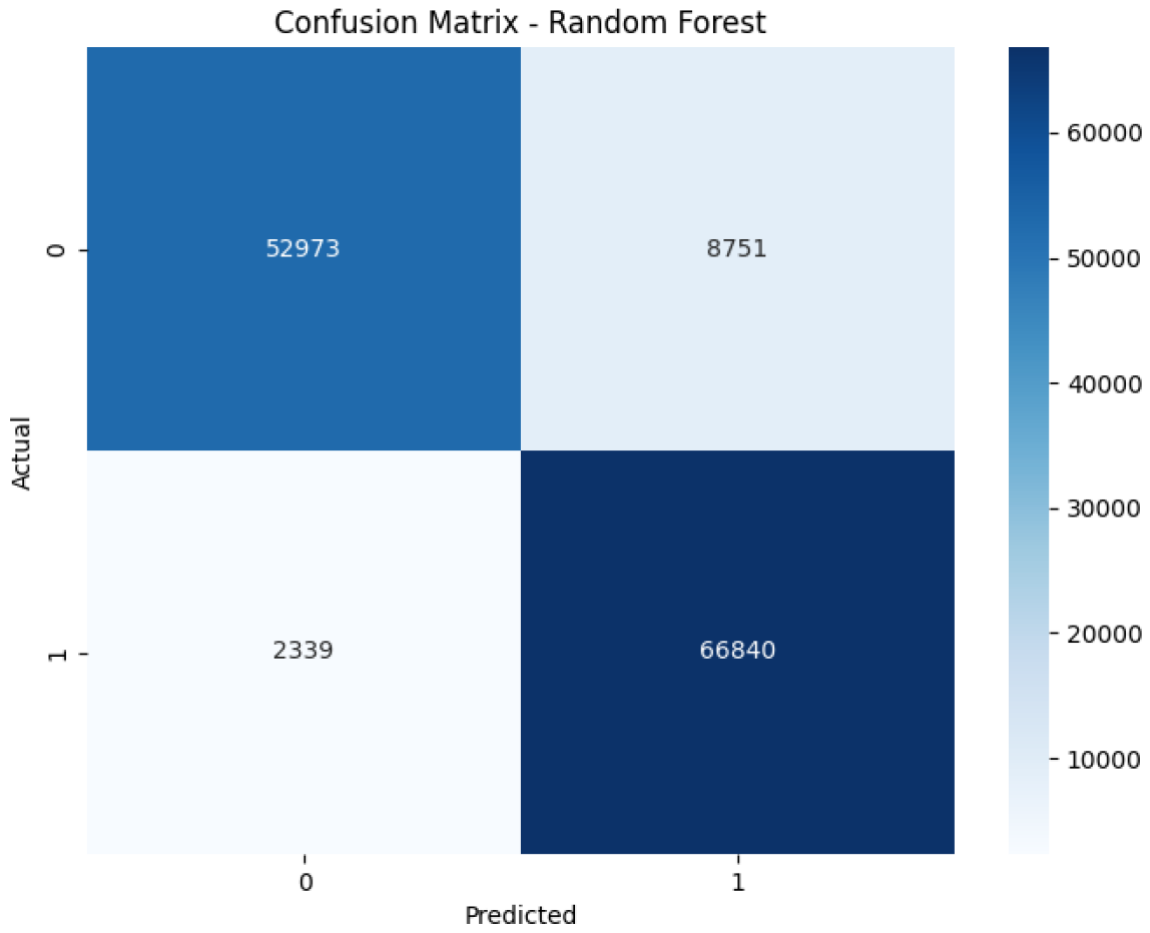


Figure 11: URL model Confussion matrix

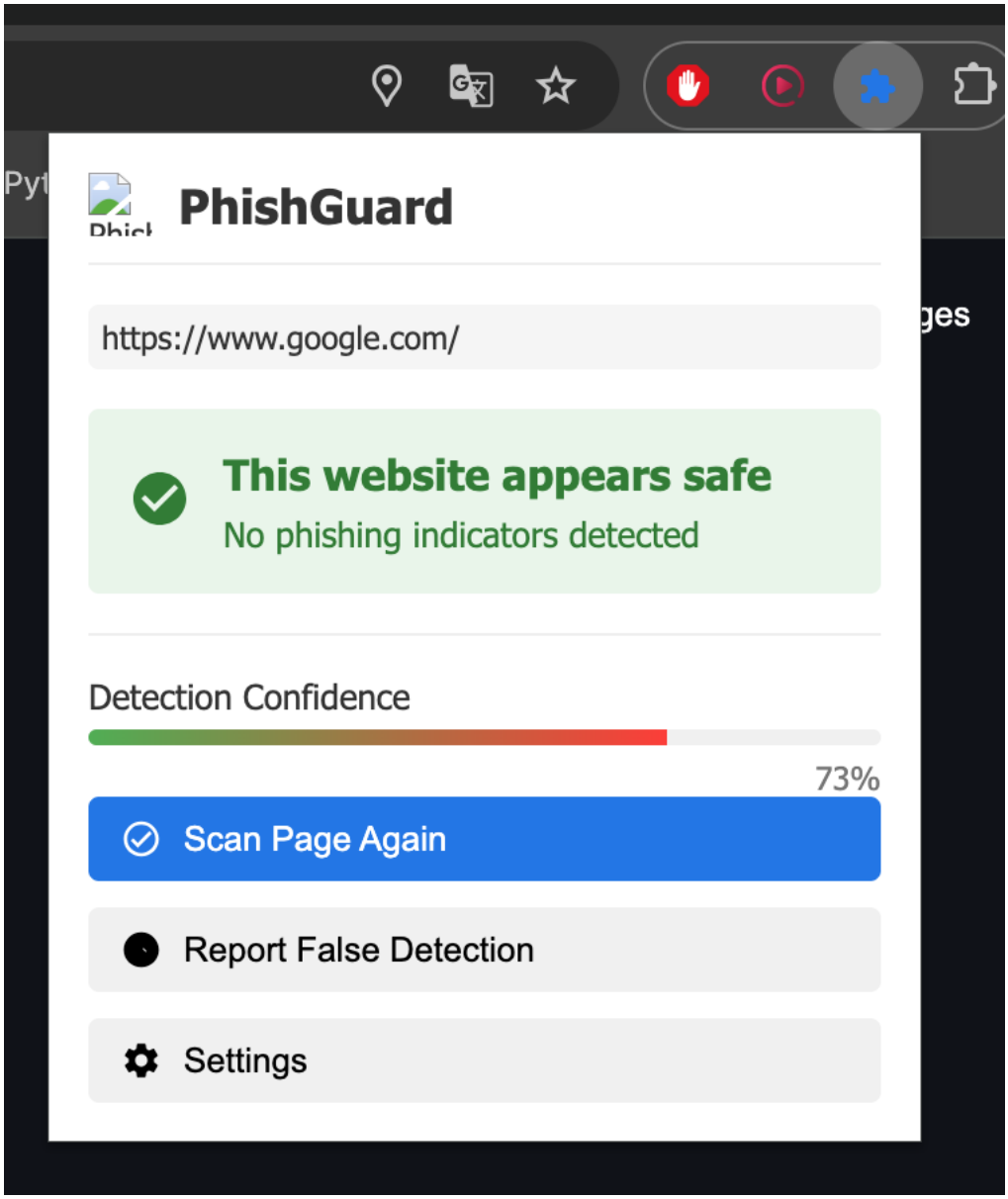


Figure 12: PhishGuard popup window

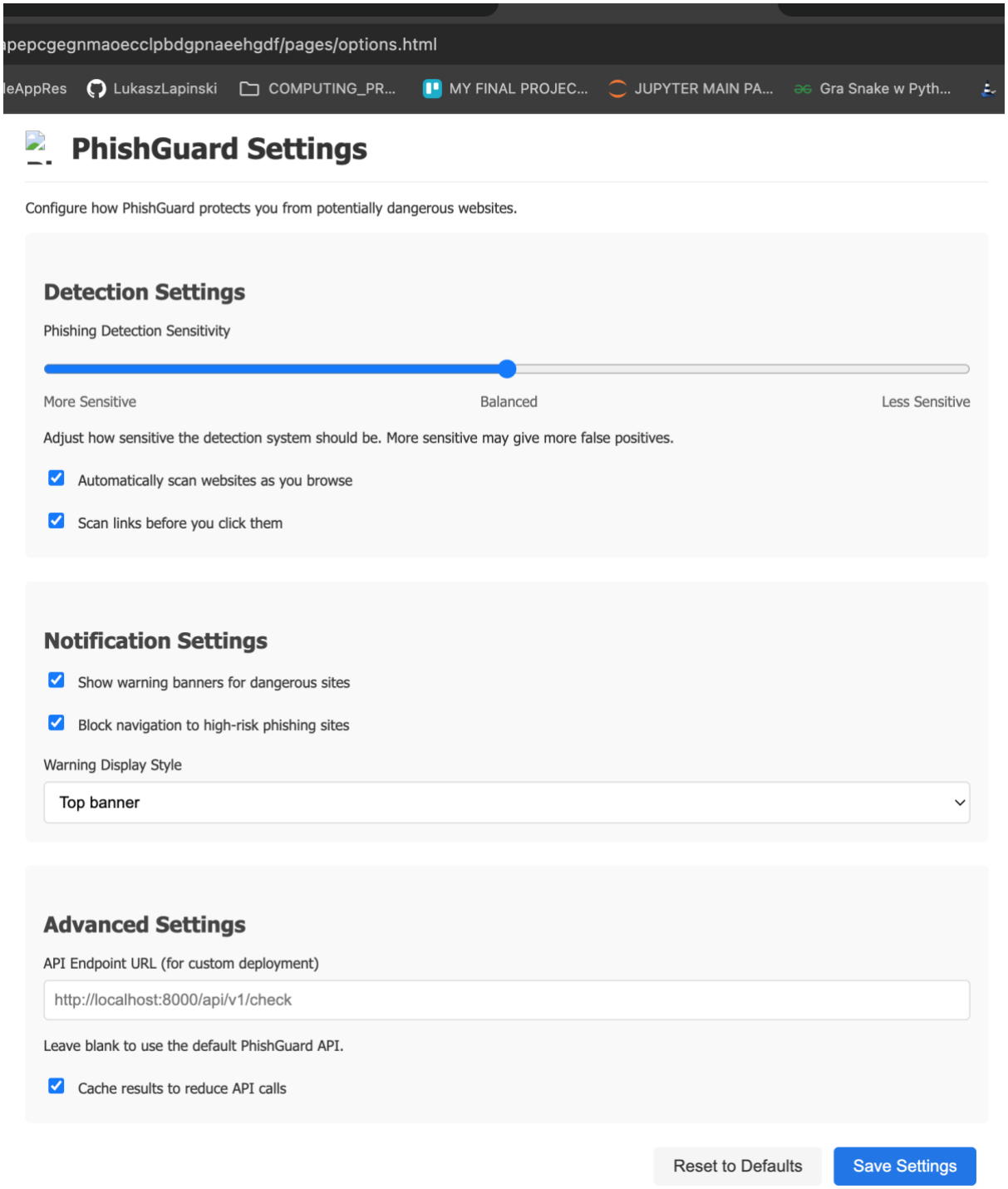


Figure 13: PhishGuard settings window

8. Reflection

The development of the PhishGuard project was both challenging and rewarding, offering numerous opportunities for technical growth and critical self-assessment. Reflecting on the process reveals several key insights and areas for future improvement.

8.1. Successes and Strengths

One of the major successes of the project was the creation of a robust, real-time phishing detection system that is both modular and extensible. The use of a well-engineered URL-based model provided strong performance and reliability, making it suitable for deployment in a browser extension. The decision to use industry-standard tools such as Google Colab, FastAPI, and VS Code facilitated rapid development, reproducibility, and ease of collaboration. The modular architecture of the backend and extension also ensures that the system can be expanded in the future as new data sources or detection techniques become available.

The project also provided valuable experience in end-to-end machine learning system development, from data acquisition and cleaning, through feature engineering and model training, to deployment and user interface integration. The iterative, research-driven approach allowed for continuous learning and adaptation in response to new challenges.

8.2. Challenges and Areas for Improvement

One of the main challenges during the development process was the lack of suitable datasets. After spending several hours searching for appropriate data for both content-based and metadata-based models, I ultimately decided to adjust the project's scope and scrape some data on my own. However, the scraping process turned out to be both time-consuming and largely ineffective. The collected data had very limited features and was highly imbalanced—approximately 90% legitimate sites and only 10% phishing sites.

Another significant issue, which ultimately became a deal-breaker, was the inconsistency between different Python environments. After successfully scraping the data and training the models in Google Colab, I was unable to run the same setup on my local machine using VS Code. The problem stemmed from numerous incompatible dependencies, which made it impossible to complete the implementation of the content-based and metadata-based models. As a result, I decided to focus solely on the URL-based model.

8.3. Lessons Learned

The project underscored the importance of flexibility and pragmatism in real-world machine learning development. While ambitious, multi-model architectures are appealing in theory, practical constraints often necessitate a more focused approach. Thorough data validation, modular system design, and clear documentation were critical to maintaining progress and ensuring reproducibility.

The experience also highlighted the value of starting with a simple, robust baseline and iteratively building complexity only as data and resources allow. This approach not only accelerates development but also ensures that a functional product can be delivered even if unforeseen challenges arise.

8.4. Implications for Future Work

Looking ahead, several avenues for improvement and expansion are clear. Securing access to more current and diverse phishing datasets—potentially through partnerships with security organizations or real-time data feeds—would enable the development and deployment of more sophisticated models. Further automation and standardization of the data pipeline would enhance scalability and reproducibility. Finally,

ongoing user feedback and real-world deployment data will be invaluable for refining the model and extending the system's capabilities.

9. Conclusion

The PhishGuard project set out to develop a robust, real-time phishing detection system capable of protecting users from a wide range of online threats. Through a combination of research, experimentation, and iterative engineering, the project delivered a modular and extensible solution cantered on a highly effective URL-based machine learning model.

While the initial vision included the integration of content-based and metadata-based models, practical challenges—particularly around data availability and system complexity—necessitated a more focused approach. The final product, though streamlined, demonstrates strong performance, reliability, and real-world applicability, providing immediate value to users through a browser extension and API.

The project's journey highlighted the importance of data quality, modular design, and adaptability in the face of unforeseen obstacles. The lessons learned and the infrastructure established lay a solid foundation for future enhancements, including the potential integration of additional data sources and more advanced detection techniques as resources and data permit.

In summary, PhishGuard stands as a testament to pragmatic, research-driven development in cybersecurity, offering both immediate protection and a platform for ongoing innovation in phishing detection.

References:

- Google. (n.d.). *Getting started - Chrome Developers*. Chrome Developers. https://developer.chrome.com/docs/extensions/get-started

- Python Software Foundation. (n.d.). *Python documentation*. https://www.python.org/doc/

- DevDocs. (n.d.). *JavaScript documentation*. https://devdocs.io/javascript/

- Mendeley Data. (2022). *Phishing Dataset*. https://data.mendeley.com/datasets/vfszbj9b36/1

- Dua, D., & Graff, C. (n.d.). *Phishing Websites Data Set*. UCI Machine Learning Repository. https://archive.ics.uci.edu/dataset/327/phishing+websites

- Harisudhan411. (n.d.). *Phishing and Legitimate URLs*. Kaggle. https://www.kaggle.com/datasets/harisudhan411/phishing-and-legitimate-urls?resource=download

- Google. (n.d.). *Safe Browsing – Google Transparency Report*. https://safebrowsing.google.com/

- Netcraft. (n.d.). *Browser extension*. https://www.netcraft.com/apps-extensions/browser-extension/

- PhishTank. (n.d.). *PhishTank – Join the fight against phishing*. https://phishtank.org/

- Google. (n.d.). *Logistic Regression: Crash Course*. Google Developers. https://developers.google.com/machine-learning/crash-course/logistic-regression

- Built In. (2023, June 1). *What is the Random Forest Algorithm?*. https://builtin.com/data-science/random-forest-algorithm

- Built In. (2023, March 14). *What is an Ensemble Model?*. https://builtin.com/machine-learning/ensemble-model

- Alazab, M., Awajan, A., Mesleh, A. M., Alazab, A., Abraham, A., & Gharghan, S. K. (2021). *Intelligent phishing detection based on deep learning techniques: A comprehensive survey*. *Artificial Intelligence Review, 55*, 1025–1074. https://doi.org/10.1007/S10462-021-09976-0

- Kiruthika, R., & Priyadharsini, R. (2021). *Detection of phishing websites using machine learning: A review*. *SN Computer Science, 2*(4). https://doi.org/10.1007/s42979-021-00557-0

- Tiangolo, S. (n.d.). *FastAPI documentation*. https://fastapi.tiangolo.com/

- Google. (n.d.). *Manifest V3 migration guide*. Chrome Developers. https://developer.chrome.com/docs/extensions/develop/migrate/what-is-mv3

- GitHub, Inc. (n.d.). *GitHub*. https://github.com/